

Technical Report

PUT AND DELETE APR IMPLEMENTATION:

```
const express = require('express');
const router = express.Router();

let attendanceRecords = [
  { id: "1", studentId: "STU-123", date: "2026-05-09", presenceStatus: "Absent" },
  { id: "2", studentId: "STU-456", date: "2026-05-09", presenceStatus: "Present" }
];

router.put('/api/attendance/:id', (req, res) => {
  const recordId = req.params.id;
  const { studentId, date, presenceStatus } = req.body;

  if (!studentId || !date || !presenceStatus) {
    return res.status(400).json({
      error: "Bad Request",
      message: "Missing required fields: studentId, date, presenceStatus"
    });
  }

  const recordIndex = attendanceRecords.findIndex(record => record.id === recordId);

  if (recordIndex === -1) {
    return res.status(404).json({
      error: "Not Found",
      message: `No attendance record found with ID: ${recordId}`
    });
  }

  attendanceRecords[recordIndex] = {
    id: recordId,
    studentId,
    date,
  }
```

```
    presenceStatus
  };

  return res.status(200).json({
    message: "Attendance record updated successfully",
    data: attendanceRecords[recordIndex]
  });
});

router.delete('/api/attendance/:id', (req, res) => {
  const recordId = req.params.id;

  const recordIndex = attendanceRecords.findIndex(record => record.id === recordId);

  if (recordIndex === -1) {
    return res.status(404).json({
      error: "Not Found",
      message: `No attendance record found with ID: ${recordId} to delete`
    });
  }

  attendanceRecords.splice(recordIndex, 1);

  return res.status(200).json({
    message: "Attendance record deleted successfully",
    deletedId: recordId
  });
});

module.exports = router;
```

openapi: 3.0.0

info:

title: Attendance & Presence Service API

version: 1.0.0

paths:

/api/attendance/{id}:

put:

summary: Update an existing attendance status

description: Used to update an existing status using the Student ID, Date, and Presence Status.

parameters:

- in: path

name: id

required: true

schema:

type: string

description: The unique identifier of the attendance record.

requestBody:

required: true

content:

application/json:

schema:

type: object

required:

- studentId

- date

- presenceStatus

properties:

studentId:

type: string

example: "STU-98765"

date:

type: string

format: date

example: "2026-05-09"

presenceStatus:

type: string

example: "Present"

responses:

'200':

description: Attendance record updated successfully

'400':

description: Bad Request - Missing required inputs

'404':

description: Not Found - Record ID does not exist

delete:

summary: Remove a mistake or duplicate entry

description: Deletes an existing attendance record by its ID.

parameters:

- in: path

name: id

required: true

schema:

type: string

description: The unique identifier of the attendance record to delete.

responses:

'200':

description: Attendance record deleted successfully

'404':

description: Not Found - Record ID does not exist

Complete OpenAPI YAML:

openapi: 3.0.0

info:

title: School Management System API

version: 1.0.0

description: >

RESTful API documentation for the Academic Grade Service and Attendance & Presence Service. Security requires authorized roles (Teacher/Admin) for grade modification. Availability is guaranteed 24/7 for attendance operations.

servers:

- url: http://localhost:8080

description: Local development server

paths:

——— GRADE SERVICE

/api/grades/{studentId}:

get:

summary: Retrieve a student's grade history

description: >

Returns the full list of grades for the given student ID.

Accessible by Students (own record), Teachers, and Admins.

Security — only authenticated users may call this endpoint.

tags:

- Grade Service

parameters:

- in: path

name: studentId

required: true

schema:

type: string

description: The unique ID of the student.

example: "STU2024001"

responses:

'200':

description: Grades retrieved successfully.

content:

application/json:

schema:

type: object

properties:

studentId:

type: string

example: "STU2024001"

grades:

type: array

items:

type: string

example:

- "Mathematics: A"
- "Physics: B+"
- "Software Engineering: A+"
- "Data Structures: A"
- "Databases: B"

'400':

description: Bad Request — studentId is null or empty.

content:

application/json:

schema:

\$ref: '#/components/schemas/ErrorResponse'

example:

error: "Bad Request"

message: "studentId must not be null or empty."

'401':

description: Unauthorized — user is not authenticated.

content:

application/json:

schema:

\$ref: '#/components/schemas/ErrorResponse'

example:

error: "Unauthorized"

message: "Authentication required to access grade records."

'404':

description: Not Found — no grade records exist for this student.

content:

application/json:

schema:

\$ref: '#/components/schemas/ErrorResponse'

example:

error: "Not Found"

message: "No grades found for student ID: STU2024001"

/api/grades:

post:

summary: Create a new grade record

description: >

Allows a Teacher or Admin to post a new grade entry for a student.

Security constraint — only users with role "Teacher" or "Admin" are authorized to call this endpoint. Students cannot post grades.

Constraint — once a semester is officially closed, grades can no longer be modified or created through this endpoint.

tags:

- Grade Service

requestBody:

required: true

content:

application/json:

schema:

type: object

required:

- studentId
- courseName
- grade

properties:

studentId:

type: string

example: "STU2024001"

courseName:

type: string

example: "Software Engineering"

grade:

type: string

example: "A+"

exportFormat:
type: string
enum: [CSV, PDF, PLAIN_TEXT]
description: Optional. Format to export the grade report.
example: "PDF"

responses:

'201':

description: Grade record created successfully.

content:

application/json:

schema:

type: object

properties:

message:

type: string

example: "Grade record created successfully."

data:

type: object

properties:

studentId:

type: string

example: "STU2024001"

courseName:

type: string

example: "Software Engineering"

grade:

type: string

example: "A+"

'400':

description: Bad Request — missing required fields.

content:

application/json:

schema:

\$ref: '#/components/schemas/ErrorResponse'

example:

error: "Bad Request"

message: "Missing required fields: studentId, courseName, grade."

'401':

description: Unauthorized — user is not authenticated.

content:

```
application/json:
  schema:
    $ref: '#/components/schemas/ErrorResponse'
  example:
    error: "Unauthorized"
    message: "Authentication required."
'403':
  description: Forbidden — students cannot create grade records.
  content:
    application/json:
      schema:
        $ref: '#/components/schemas/ErrorResponse'
      example:
        error: "Forbidden"
        message: "Only Teacher or Admin roles can create grade records."
'423':
  description: Locked — semester is closed, grades cannot be modified.
  content:
    application/json:
      schema:
        $ref: '#/components/schemas/ErrorResponse'
      example:
        error: "Locked"
        message: "Semester is officially closed. Grade modification is not allowed."
```

— ATTENDANCE SERVICE

```
/api/attendance/{id}:
  put:
    summary: Update an existing attendance record
    description: >
      Updates an attendance entry by its record ID.
      Availability — this endpoint is available 24/7 to support
      morning check-ins and late-night report processing.
      Constraint — a student cannot be marked Present more than
      once for the same class on the same day.
    tags:
      - Attendance Service
    parameters:
```

- in: path
 - name: id
 - required: true
 - schema:
 - type: string
 - description: The unique ID of the attendance record.
 - example: "1"
- requestBody:
 - required: true
 - content:
 - application/json:
 - schema:
 - type: object
 - required:
 - studentId
 - date
 - presenceStatus
 - properties:
 - studentId:
 - type: string
 - example: "STU-98765"
 - date:
 - type: string
 - format: date
 - example: "2026-05-09"
 - presenceStatus:
 - type: string
 - enum: [Present, Absent]
 - example: "Present"
 - responses:
 - '200':
 - description: Attendance record updated successfully.
 - content:
 - application/json:
 - schema:
 - type: object
 - properties:
 - message:
 - type: string
 - example: "Attendance record updated successfully"

```
    data:
      $ref: '#/components/schemas/AttendanceRecord'
'400':
  description: Bad Request — missing required fields or duplicate Present entry.
  content:
    application/json:
      schema:
        $ref: '#/components/schemas/ErrorResponse'
      example:
        error: "Bad Request"
        message: "Missing required fields: studentId, date, presenceStatus."
'404':
  description: Not Found — record ID does not exist.
  content:
    application/json:
      schema:
        $ref: '#/components/schemas/ErrorResponse'
      example:
        error: "Not Found"
        message: "No attendance record found with ID: 1"
```

```
delete:
  summary: Delete an attendance record
  description: >
    Removes a mistake or duplicate attendance entry by its record ID.
    Availability — available 24/7.
  tags:
    - Attendance Service
  parameters:
    - in: path
      name: id
      required: true
      schema:
        type: string
      description: The unique ID of the attendance record to delete.
      example: "1"
  responses:
    '200':
      description: Attendance record deleted successfully.
      content:
```

```
application/json:
  schema:
    type: object
    properties:
      message:
        type: string
        example: "Attendance record deleted successfully"
      deletedId:
        type: string
        example: "1"
'404':
  description: Not Found — record ID does not exist.
  content:
    application/json:
      schema:
        $ref: '#/components/schemas/ErrorResponse'
      example:
        error: "Not Found"
        message: "No attendance record found with ID: 1 to delete"
```

```
components:
  schemas:
    AttendanceRecord:
      type: object
      properties:
        id:
          type: string
          example: "1"
        studentId:
          type: string
          example: "STU-98765"
        date:
          type: string
          format: date
          example: "2026-05-09"
        presenceStatus:
          type: string
          enum: [Present, Absent]
          example: "Present"
```

ErrorResponse:

type: object

properties:

error:

type: string

message:

type: string

securitySchemes:

bearerAuth:

type: http

scheme: bearer

bearerFormat: JWT

security:

- bearerAuth: []

Postman JSON Collection:

```
{
  "info": {
    "name": "School Management System API",
    "schema": "https://schema.getpostman.com/json/collection/v2.1.0/collection.json"
  },
  "item": [
    {
      "name": "Grade Service",
      "item": [
        {
          "name": "GET /api/grades/{studentId} — Success",
          "request": {
            "method": "GET",
            "url": "http://localhost:8080/api/grades/STU2024001"
          },
          "response": [
            {
              "name": "200 OK",
              "status": "OK",
              "code": 200,
              "body": "{\n  \"studentId\": \"STU2024001\",\n  \"grades\": [\n    \"Mathematics: A\",\n    \"Physics: B+\",\n    \"Software Engineering: A+\",\n    \"Data Structures: A\",\n    \"Databases: B\"\n  ]\n}"
            }
          ]
        }
      ]
    },
    {
      "name": "GET /api/grades/{studentId} — 404 Not Found",
      "request": {
        "method": "GET",
        "url": "http://localhost:8080/api/grades/STU9999"
      },
      "response": [
        {
          "name": "404 Not Found",
          "status": "Not Found",
          "code": 404,
```

```

        "body": "{\n  \"error\": \"Not Found\", \n  \"message\": \"No grades found for student ID: STU9999\"\n}"
      }
    ],
  },
  {
    "name": "POST /api/grades — Success",
    "request": {
      "method": "POST",
      "url": "http://localhost:8080/api/grades",
      "header": [
        { "key": "Content-Type", "value": "application/json" }
      ],
      "body": {
        "mode": "raw",
        "raw": "{\n  \"studentId\": \"STU2024001\", \n  \"courseName\": \"Software Engineering\", \n  \"grade\": \"A+\", \n  \"exportFormat\": \"PDF\"\n}"
      }
    },
    "response": [
      {
        "name": "201 Created",
        "status": "Created",
        "code": 201,
        "body": "{\n  \"message\": \"Grade record created successfully.\", \n  \"data\": {\n    \"studentId\": \"STU2024001\", \n    \"courseName\": \"Software Engineering\", \n    \"grade\": \"A+\"\n  }\n}"
      }
    ]
  },
  {
    "name": "POST /api/grades — 403 Forbidden (Student role)",
    "request": {
      "method": "POST",
      "url": "http://localhost:8080/api/grades",
      "header": [
        { "key": "Content-Type", "value": "application/json" }
      ],
      "body": {
        "mode": "raw",

```

```

        "raw": "{\n  \"studentId\": \"STU2024001\", \n  \"courseName\": \"Mathematics\", \n  \"grade\": \"A\" \n}"
      },
      "response": [
        {
          "name": "403 Forbidden",
          "status": "Forbidden",
          "code": 403,
          "body": "{\n  \"error\": \"Forbidden\", \n  \"message\": \"Only Teacher or Admin roles can create grade records.\" \n}"
        }
      ]
    },
    {
      "name": "POST /api/grades — 400 Bad Request (missing fields)",
      "request": {
        "method": "POST",
        "url": "http://localhost:8080/api/grades",
        "header": [
          { "key": "Content-Type", "value": "application/json" }
        ],
        "body": {
          "mode": "raw",
          "raw": "{\n  \"studentId\": \"STU2024001\" \n}"
        }
      },
      "response": [
        {
          "name": "400 Bad Request",
          "status": "Bad Request",
          "code": 400,
          "body": "{\n  \"error\": \"Bad Request\", \n  \"message\": \"Missing required fields: studentId, courseName, grade.\" \n}"
        }
      ]
    }
  ]
},
{

```

```

"name": "Attendance Service",
"item": [
  {
    "name": "PUT /api/attendance/{id} — Success",
    "request": {
      "method": "PUT",
      "url": "http://localhost:8080/api/attendance/1",
      "header": [
        { "key": "Content-Type", "value": "application/json" }
      ],
      "body": {
        "mode": "raw",
        "raw": "{\n  \"studentId\": \"STU-98765\", \n  \"date\": \"2026-05-09\", \n
\"presenceStatus\": \"Present\" \n}"
      }
    },
    "response": [
      {
        "name": "200 OK",
        "status": "OK",
        "code": 200,
        "body": "{\n  \"message\": \"Attendance record updated successfully\", \n
\"data\": {\n    \"id\": \"1\", \n    \"studentId\": \"STU-98765\", \n    \"date\": \"2026-05-09\", \n
\"presenceStatus\": \"Present\" \n  } \n}"
      }
    ]
  },
  {
    "name": "PUT /api/attendance/{id} — 404 Not Found",
    "request": {
      "method": "PUT",
      "url": "http://localhost:8080/api/attendance/999",
      "header": [
        { "key": "Content-Type", "value": "application/json" }
      ],
      "body": {
        "mode": "raw",
        "raw": "{\n  \"studentId\": \"STU-98765\", \n  \"date\": \"2026-05-09\", \n
\"presenceStatus\": \"Present\" \n}"
      }
    }
  }
]

```

```
    },
    "response": [
      {
        "name": "404 Not Found",
        "status": "Not Found",
        "code": 404,
        "body": "{\n  \"error\": \"Not Found\", \n  \"message\": \"No attendance record found with ID: 999\"\n}"
      }
    ]
  },
  {
    "name": "PUT /api/attendance/{id} — 400 Bad Request (missing fields)",
    "request": {
      "method": "PUT",
      "url": "http://localhost:8080/api/attendance/1",
      "header": [
        { "key": "Content-Type", "value": "application/json" }
      ],
      "body": {
        "mode": "raw",
        "raw": "{\n  \"studentId\": \"STU-98765\"\n}"
      }
    },
    "response": [
      {
        "name": "400 Bad Request",
        "status": "Bad Request",
        "code": 400,
        "body": "{\n  \"error\": \"Bad Request\", \n  \"message\": \"Missing required fields: studentId, date, presenceStatus.\"\n}"
      }
    ]
  },
  {
    "name": "DELETE /api/attendance/{id} — Success",
    "request": {
      "method": "DELETE",
      "url": "http://localhost:8080/api/attendance/1"
    },
  },
```

```
"response": [
  {
    "name": "200 OK",
    "status": "OK",
    "code": 200,
    "body": "{\n  \"message\": \"Attendance record deleted successfully\", \n
  \"deletedId\": \"1\"\n}"
  },
],
{
  "name": "DELETE /api/attendance/{id} — 404 Not Found",
  "request": {
    "method": "DELETE",
    "url": "http://localhost:8080/api/attendance/999"
  },
  "response": [
    {
      "name": "404 Not Found",
      "status": "Not Found",
      "code": 404,
      "body": "{\n  \"error\": \"Not Found\", \n  \"message\": \"No attendance record
found with ID: 999 to delete\"\n}"
    }
  ]
}
]
```

NFR table:

	<u>NFR</u>	<u>How it's covered in the API docs</u>
<u>Security</u>		POST /api/grades returns 403 Forbidden for Student role; 401 Unauthorized for unauthenticated users; JWT Bearer auth defined in securitySchemes
<u>Availability (24/7)</u>		PUT and DELETE /api/attendance explicitly documented as available 24/7 in their descriptions
<u>Constraint — no duplicate Present</u>		Documented as a 400 Bad Request case in PUT /api/attendance description
<u>Constraint — closed semester</u>		Documented as 423 Locked response on POST /api/grades

Service Engineering Application

1. Service Candidate Identification

In the transition to a Service-Oriented Architecture (SOA), we have identified two high-value service candidates that provide the most utility to our School Management System:

Candidate A: Academic Grade Service

What it does?

The Academic Grade Service is the primary engine for managing student performance data. It acts as a digital ledger where instructors can securely record assessment results, midterms, and final exam marks. Instead of having grades scattered across different spreadsheets, this service centralizes everything into one secure location.

- **Justification for Selection: * Reusability across processes:** This service is the most heavily reused component in our ecosystem. It serves the Student Portal for grade viewing, the Teacher Console for grade entry, and the Administrative Module for generating official transcripts.
 - **Independence & External Potential:** The service is designed to be completely decoupled from our specific database technology. It has high potential for external usage, such as allowing a centralized national database to verify student results through a secure connection.
- **Service Classification: Business Service.** It implements core school logic rather than general-purpose technical tasks.

How it works in the system

1-For Teachers: It provides a simple way to "POST" new grades or "PUT" updates if a student's score needs correction after a regrade.

2-For Students: It allows them to "GET" their specific results instantly through the student portal as soon as they are published.

3-Key Advantage: It is a Business Service because it handles the school's most sensitive data—academic records—and ensures that only authorized staff can make changes, which supports our overall Security goals.

Candidate B: Attendance & Presence Service

What it does?

The Attendance & Presence Service is a dedicated tool for tracking student engagement and physical presence in the classroom. It moves beyond simple paper rosters by creating a real-time record of who is in class, which is vital for both academic success and school safety.

- **Justification for Selection:**
 - **Reusability across processes:** Attendance data is required by multiple departments. For example, the Exam Eligibility module uses it to check if a student met the attendance threshold, and the Security Module uses it to verify who is currently on campus.
 - **Independence & External Potential:** It is an independent "plug-and-play" module. This allows us to integrate it with external hardware, such as student ID card scanners or mobile apps, without needing to modify the rest of the software.
- **Service Classification: Business Service.**

How it works in the system

1-Daily Tracking: Every morning or at the start of a lecture, the system creates a new entry for each student.

2-Calculations: The service doesn't just store dates; it automatically calculates a student's total attendance percentage. For example, if a student misses too many classes, this service flags them as "at risk."

3-Key Advantage: This is also a Business Service because tracking attendance is a core requirement of the educational process. It is highly Reusable because it can be connected to mobile apps or future hardware, like card scanners, without needing to change the main software.

2. Service Requirements Specification

A) Grade Service Specification

- **Functional Requirements:** The service must allow teachers to enter and update marks and allow students to view their specific academic transcripts.
- **Non-Functional Requirements:**
 - **Security:** Only authorized users with "Teacher" or "Admin" status can modify grade data.
 - **Performance:** The service must be optimized to handle hundreds of students checking results at the same time without slowing down.
- **Inputs/Outputs & Constraints:**
 - **Inputs:** Student ID, Course Name, and the numeric Grade.
 - **Outputs:** A standardized digital grade report.
 - **Constraints:** Once a semester is officially closed, grades can no longer be modified through the service.

B) Attendance Service Specification

- **Functional Requirements:** The service must log daily presence and provide cumulative totals for every student.
- **Non-Functional Requirements:**
 - **Availability:** The service must be available 24/7 to ensure morning check-ins and late-night reports can be processed without interruption.
- **Inputs/Outputs & Constraints:**
 - **Inputs:** Student ID, Date, and Presence Status.
 - **Outputs:** An attendance summary for a specific student or class.
 - **Constraints:** A student cannot be marked "Present" more than once for the same class on the same day.

3. Service Interface Design (The Roadmap)

I have designed these services as RESTful APIs:

- **Endpoints & HTTP Mapping:**
 - **GET /api/grades/{studentID}:** Used by Sayed to retrieve a student's history.
 - **POST /api/grades:** Used by Sayed to create a new record.
 - **PUT /api/attendance/{id}:** Used by Hany to update an existing status.
 - **DELETE /api/attendance/{id}:** Used by Hany to remove a mistake or duplicate entry.
 - **Data Formats:** All communication will be handled using JSON, ensuring that the data is easy to read and compatible with any device.
-

4. Service Composition Workflow

- **Service Name:** "Final Student Evaluation Workflow".
- **Workflow Explanation:** 1. The workflow begins by calling the Attendance Service to see if a student has enough presence to be eligible for final grades. 2. If eligible, it then automatically calls the Grade Service to pull all their scores for the term. 3. Finally, it combines this information into one final "Completion Report" that is sent to the student.

Final Student Evaluation Workflow

1. Workflow Overview

The “Final Student Evaluation Workflow” is a composite service in the School Management System that combines multiple independent services to evaluate a student’s final academic status.

The workflow integrates:

- Attendance & Presence Service
- Grade Service

The final result is a unified Completion Report that contains:

- Attendance eligibility
- Academic grades
- Final completion decision

2. Participating Services

Service	Responsibility
Attendance Service	Checks attendance eligibility
Grade Service	Retrieves academic grades
Final Evaluation Workflow Combines results into one final report	

3. Step-by-Step Workflow Logic

Step 1 — Attendance Verification

The workflow starts by calling the Attendance Service.

Process

- The system receives a Student ID.
- The Attendance Service calculates attendance percentage.
- The service checks whether the student satisfies the required attendance threshold.

Example Rule

Minimum Attendance Requirement = 75%

Output

- Eligible
 - Not Eligible
-

Step 2 — Grade Retrieval

If the student is eligible:

- The workflow calls the Grade Service.

Process

The Grade Service retrieves:

- Assignment grades
- Midterm grades
- Final exam grades

Output

A complete academic transcript for the semester.

Step 3 — Completion Report Generation

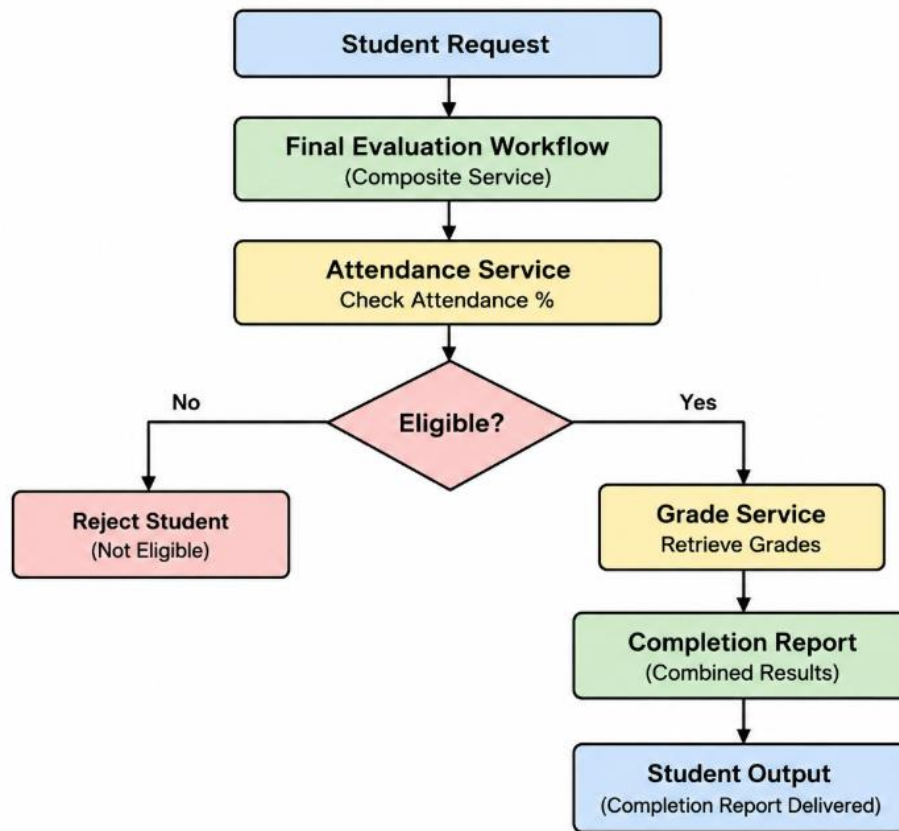
The workflow combines:

- Attendance result
- Academic grades
- Final evaluation status

Final Output

Completion Report

4. Service Composition Diagram



5. Architectural Justification

This workflow follows Service-Oriented Architecture principles because:

- Each service is independent and reusable
- Services communicate through clear interfaces
- The workflow supports scalability and maintainability
- Additional services can be integrated easily in the future

6. Conclusion

The “Final Student Evaluation Workflow” demonstrates how multiple services collaborate to automate the academic evaluation process efficiently. By combining attendance verification and grade retrieval, the system generates accurate completion reports while maintaining modularity, scalability, and maintainability.